LECTURE – ABSTRACT AND INTERFACE
OOP (JAVA)



MR. MUHAMMAD SALEEM
LECTURER
DEPARTMENT OF COMPUTER SCIENCE

Dawood University of Engineering and Technology, Karachi

AGENDA

- INTRODUCTION TO ABSTRACT
- ABSTRACT CLASS
- ABSTRACT METHOD
- EXAMPLES
- INTRODUCTION OF INTERFACE
- INTERFACE EXAMPLE
- DIFFERENCE BETWEEN ITERFACE AND ABSTRACT

ABSTRACT

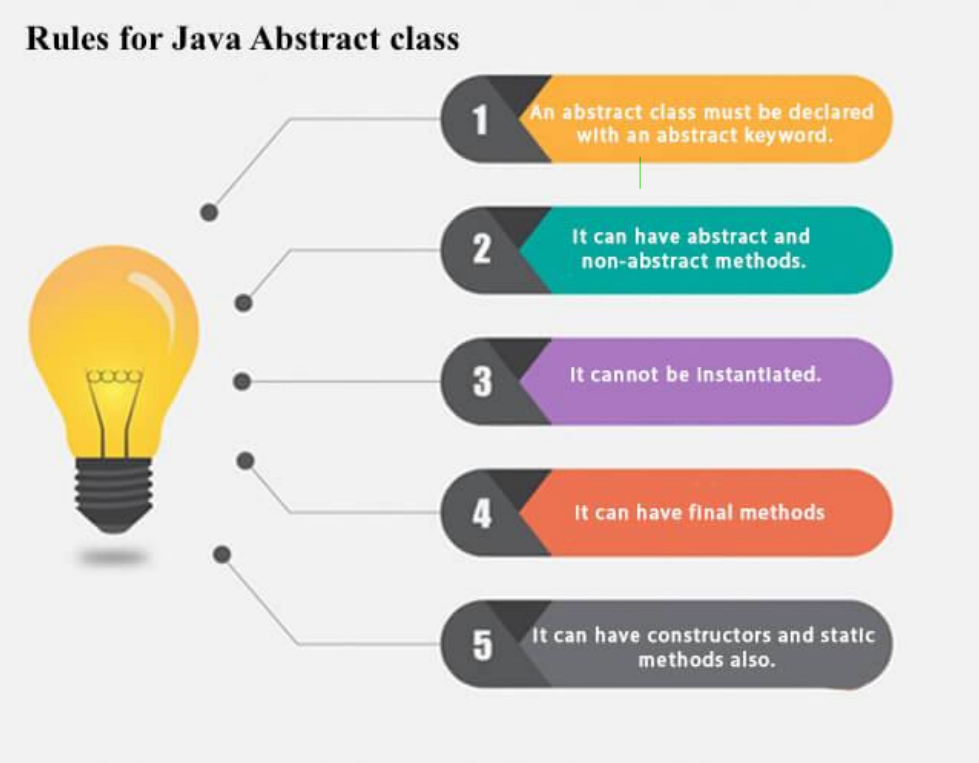
- According to Google: existing in thought or as an idea but not having a physical or concrete existence.
- **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.
- Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.
- Abstraction lets you focus on what the object does instead of how it does it.

ABSTRACT CLASS IN JAVA

- A class which is declared with the abstract keyword is known as an abstract class in Java.
- It can have abstract and non-abstract methods (method with the body).
- Before learning the Java abstract class, let's understand the abstraction in Java first.
- It needs to be extended and its method implemented.
- It cannot be instantiated.

POINTS TO REMEMBER

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.



ABSTRACT METHODS IN JAVA

- A method which is declared as abstract and does not have implementation is known as an abstract method.

```
abstract class Bike{
  abstract void run();
}
class Honda4 extends Bike{
void run(){System.out.println("running safely");}
public static void main(String args[]){
 Bike obj = new Honda4();
 obj.run();
}
}
```

```
abstract class Shape{
abstract void draw();
}
//In real scenario, implementation is provided by other
s i.e. unknown by end user
class Rectangle extends Shape{
void draw(){System.out.println("drawing rectangle");}

}
class Circle1 extends Shape{
void draw(){System.out.println("drawing circle");}
}
//In real scenario, method is called by programmer or
user
class TestAbstraction1{
public static void main(String args[]){
Shape s=new Circle1();//In a real scenario, object is pr
ovided through method, e.g., getShape() method
```

```
//Example of an abstract class that has abstract and non-abstract method
s
 abstract class Bike{
   Bike(){System.out.println("bike is created");}
   abstract void run();
   void changeGear(){System.out.println("gear changed");}   }
//Creating a Child class which inherits Abstract class
 class Honda extends Bike{
 void run(){System.out.println("running safely..");}  }
//Creating a Test class which calls abstract and non-abstract methods
 class TestAbstraction2{
 public static void main(String args[]){
  Bike obj = new Honda();
  obj.run();
  obj.changeGear();
 }
}
```

WAYS TO ACHIEVE ABSTRACTION

- There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

INTERFACE IN JAVA

- An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.
- The interface in Java is *a mechanism to achieve abstraction*.
- There can be only abstract methods in the Java interface, not method body.
- It is used to achieve abstraction and multiple inheritance in Java.
- In other words, you can say that interfaces can have abstract methods and variables.
- It cannot have a method with the body.

JAVA INTERFACE

- Java Interface also **represents the IS-A relationship**.
- It cannot be instantiated just like the abstract class.
- Since Java 8, we can have **default and static methods** in an interface.
- Since Java 9, we can have **private methods** in an interface.

WHY USE JAVA INTERFACE?

- There are mainly three reasons to use interface. They are given below.
- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

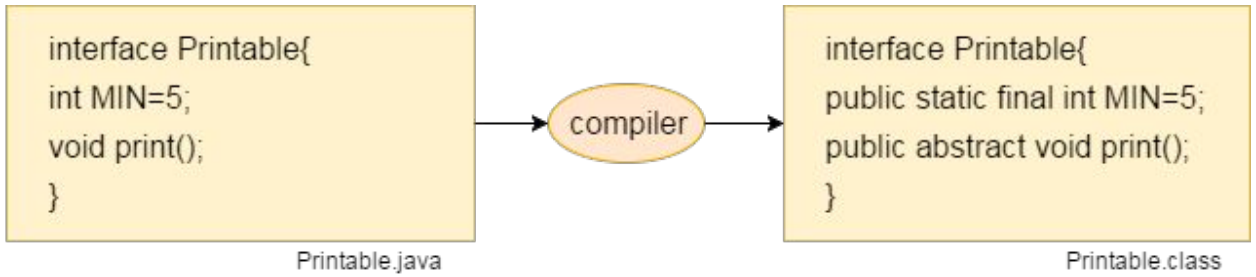


DECLARATION OF AN INTERFACE

- An interface is declared by using the interface keyword.
- It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default.
- A class that implements an interface must implement all the methods declared in the interface.

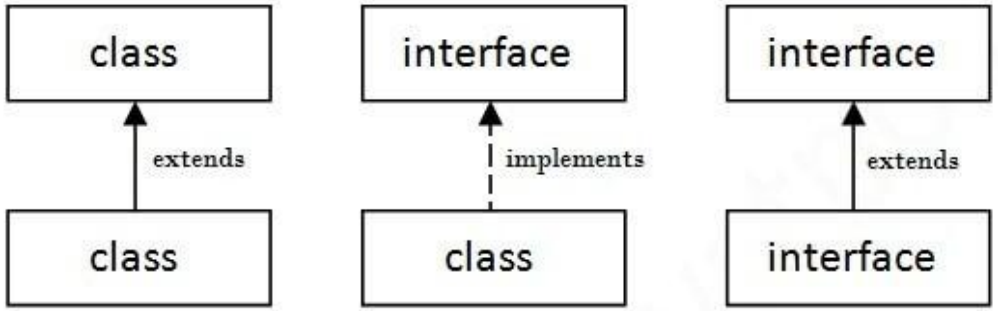
SYNTAX OF INTERFACE

- Interface fields are public, static and final by default, and the methods are public and abstract



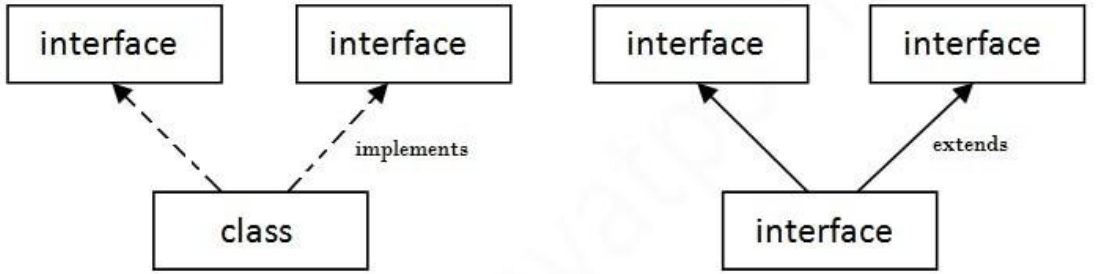
THE RELATIONSHIP BETWEEN CLASSES AND INTERFACES

- As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



MULTIPLE INHERITANCE IN JAVA BY INTERFACE

- If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

DIFFERENCE BETWEEN ABSTRACT CLASS AND INTERFACE

- Abstract class and interface both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated.
- But there are many differences between abstract class and interface that are given below.

Abstract class	Interface
1) Abstract class can **have abstract and non-abstract** methods.	Interface can have **only abstract** methods. Since Java 8, it can have **default and static methods** also.
2) Abstract class **doesn't support multiple inheritance**.	Interface **supports multiple inheritance**.
3) Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
4) Abstract class **can provide the implementation of interface**.	Interface **can't provide the implementation of abstract class**.
5) The **abstract keyword** is used to declare abstract class.	The **interface keyword** is used to declare interface.
6) An **abstract class** can extend another Java class and implement multiple Java interfaces.	An **interface** can extend another Java interface only.
7) An **abstract class** can be extended using keyword "extends".	An **interface** can be implemented using keyword "implements".
8) A Java **abstract class** can have class members like private, protected, etc.	Members of a Java interface are public by default.

Example:
public abstract class Shape{
public abstract void draw();
}
	Example:
	public interface Drawable{
	void draw();
	}



THANK YOU